

SKILL WEEK_07

1. To Store Commands, Maintain History Buffer, Handle Capacity Limits, Retrieve Entries, Display History, Validate Consistency

CODE:



```
root@SRIVALLI: ~
GNU nano 7.2 history.c *
#include <stdio.h>
#include <string.h>

#define MAX_HISTORY 5
#define MAX_CMD 100

int main()
{
    char history[MAX_HISTORY][MAX_CMD];
    char command[MAX_CMD];
    int count = 0;

    while(1)
    {
        printf("Enter Command (history/exit): ");
        fgets(command, MAX_CMD, stdin);

        command[strcspn(command, "\n")] = '\0';

        if(strcmp(command, "exit") == 0)
            break;

        if(strcmp(command, "history") == 0)
        {
            printf("\nCommand History:\n");
            for(int i = 0; i < count; i++)
                printf("%d. %s\n", i + 1, history[i]);
            continue;
        }

        if(count < MAX_HISTORY)
        {
            strcpy(history[count], command);
            count++;
        }
        else
        {
            // Handle capacity limit (not shown in image)
        }
    }
}
```

```

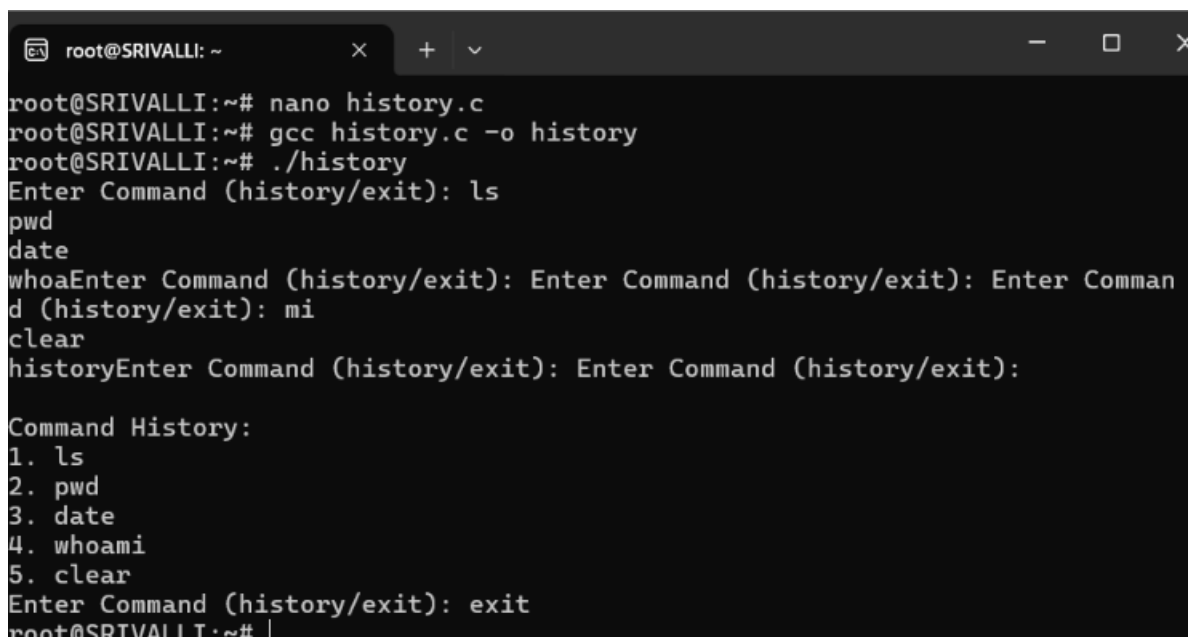
        if(count < MAX_HISTORY)
        {
            strcpy(history[count], command);
            count++;
        }
        else
        {
            for(int i = 1; i < MAX_HISTORY; i++)
                strcpy(history[i - 1], history[i]);

            strcpy(history[MAX_HISTORY - 1], command);
        }
    }

    return 0;
}

```

RESULT:



```

root@SRIVALLI: ~
root@SRIVALLI:~# nano history.c
root@SRIVALLI:~# gcc history.c -o history
root@SRIVALLI:~# ./history
Enter Command (history/exit): ls
pwd
date
whoami
Enter Command (history/exit): Enter Command (history/exit): Enter Command (history/exit): mi
clear
history
Enter Command (history/exit): Enter Command (history/exit):
Command History:
1. ls
2. pwd
3. date
4. whoami
5. clear
Enter Command (history/exit): exit
root@SRIVALLI:~#

```

REPORT:

In this experiment, a command history management program was developed to store and maintain previously entered commands. A fixed-size history buffer was used to save commands entered by the user. When the buffer reached its maximum capacity, the oldest command was removed and the newest command was added. The program allows users to retrieve and display command history using the history command. This experiment helps in understanding history buffer management, command storage, retrieval mechanisms, capacity handling, and consistency verification in shell-like environments.

2. To Parse Output Redirection, Create Output Files, Redirect stdout, Handle Permissions, Verify Output Data, Test Redirection
- CODE:

```
root@SRIVALLI: ~  
GNU nano 7.2 redirection.c *  
#include <stdio.h>  
#include <fcntl.h>  
#include <unistd.h>  
#include <stdlib.h>  
  
int main()  
{  
    int fd;  
  
    fd = open("output.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);  
  
    if(fd < 0)  
    {  
        printf("File creation failed\n");  
        return 1;  
    }  
  
    dup2(fd, STDOUT_FILENO);  
  
    printf("This output is redirected to output.txt\n");  
    printf("Operating Systems Lab - Skill 6\n");  
  
    close(fd);  
  
    return 0;  
}
```

RESULT:

```
root@SRIVALLI:~# nano redirection.c  
root@SRIVALLI:~# gcc redirection.c -o redirection  
root@SRIVALLI:~# ./redirection  
root@SRIVALLI:~# cat output.txt  
This output is redirected to output.txt  
Operating Systems Lab - Skill 6  
root@SRIVALLI:~# ls -l output.txt  
-rw-r--r-- 1 root root 72 Sep 19 03:09 output.txt  
root@SRIVALLI:~#
```

REPORT:

In this experiment, a program was developed to demonstrate output redirection in Linux using the `dup2()` system call. The program creates an output file and redirects the standard output (stdout) to that file instead of displaying data on the terminal. File permissions are set during file creation using the `open()` system call. After execution, the output data is verified by reading the contents of the file. This experiment helps in understanding file creation, output redirection, file descriptor management, and the use of system calls for handling input/output operations in Linux.

3. To Parse Append Operator, Open Files in Append Mode, Redirect Streams, Preserve Existing Data, Validate Appends, Test Scenarios

CODE:

```
root@SRIVALLI: ~
GNU nano 7.2 append_redirection.c *
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <stdlib.h>

int main()
{
    int fd;

    fd = open("output.txt", O_WRONLY | O_CREAT | O_APPEND, 0644);

    if(fd < 0)
    {
        perror("File Open Error");
        return 1;
    }

    dup2(fd, STDOUT_FILENO);

    printf("This line is appended to the file.\n");

    close(fd);

    return 0;
}
```

RESULT:

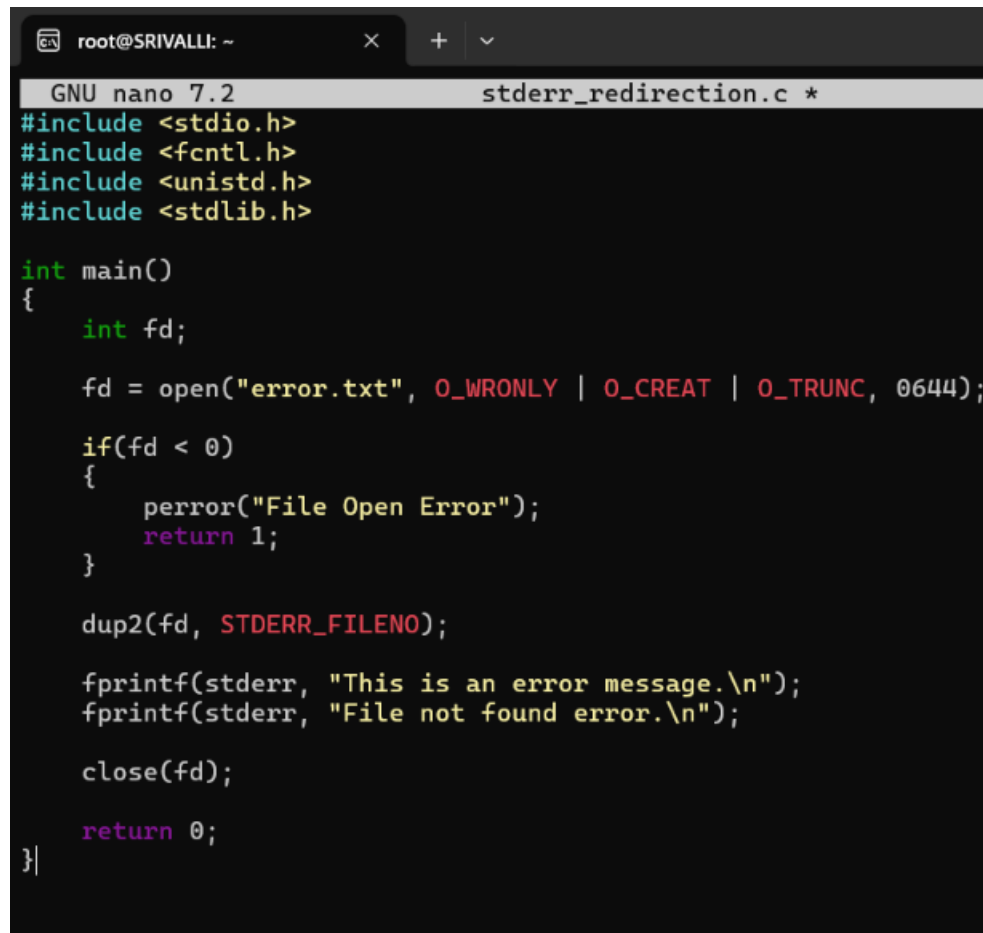
```
root@SRIVALLI:~# nano append_redirection.c
root@SRIVALLI:~# gcc append_redirection.c -o append_redirection
root@SRIVALLI:~# ./append_redirection
root@SRIVALLI:~# ./append_redirection
root@SRIVALLI:~# ./append_redirection
root@SRIVALLI:~# ./append_redirection
root@SRIVALLI:~# cat output.txt
This output is redirected to output.txt
Operating Systems Lab - Skill 6
This line is appended to the file.
This line is appended to the file.
This line is appended to the file.
This line is appended to the file.
This line is appended to the file.
root@SRIVALLI:~# ls -l output.txt
-rw-r--r-- 1 root root 247 Sep 19 03:13 output.txt
```

REPORT:

In this experiment, a program was developed to demonstrate append output redirection in Linux. The program opens a file in append mode using the `O_APPEND` flag and redirects the standard output stream to the file using the `dup2()` system call. Unlike normal redirection, append mode preserves the existing contents of the file and adds new output at the end. The appended data is verified by displaying the file contents after multiple executions. This experiment helps in understanding append operations, file handling, output stream redirection, and data preservation in Linux systems.

4. TO parse stderr Redirection, Redirect Error Streams, Capture Error Output, Verify File Contents, Handle Failures, Test Error Cases

CODE:



```
GNU nano 7.2 stderr_redirection.c *
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <stdlib.h>

int main()
{
    int fd;

    fd = open("error.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if(fd < 0)
    {
        perror("File Open Error");
        return 1;
    }

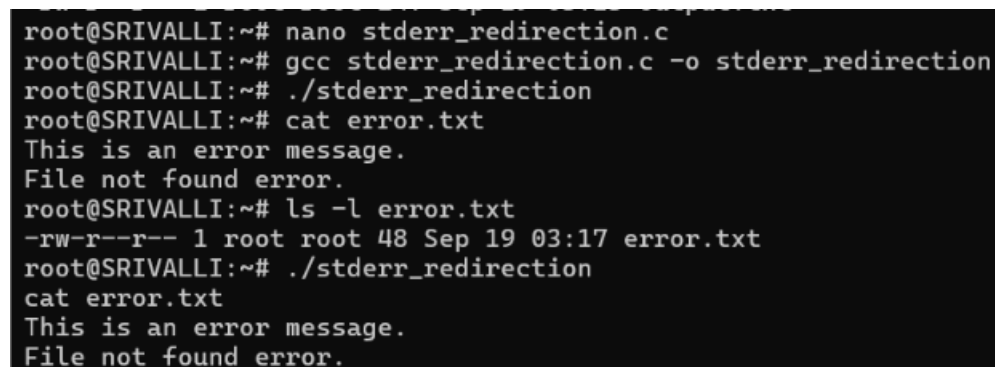
    dup2(fd, STDERR_FILENO);

    fprintf(stderr, "This is an error message.\n");
    fprintf(stderr, "File not found error.\n");

    close(fd);

    return 0;
}
```

RESULT:



```
root@SRIVALLI:~# nano stderr_redirection.c
root@SRIVALLI:~# gcc stderr_redirection.c -o stderr_redirection
root@SRIVALLI:~# ./stderr_redirection
root@SRIVALLI:~# cat error.txt
This is an error message.
File not found error.
root@SRIVALLI:~# ls -l error.txt
-rw-r--r-- 1 root root 48 Sep 19 03:17 error.txt
root@SRIVALLI:~# ./stderr_redirection
cat error.txt
This is an error message.
File not found error.
```

REPORT:

In this experiment, a program was developed to demonstrate standard error (stderr) redirection in Linux. The program creates an error file and redirects the error stream to that file using the `dup2()` system call. Error messages generated by the program are written to the file instead of being displayed on the terminal. The contents of the file are verified after execution to ensure that the error output has been captured correctly. This experiment helps in understanding error stream handling, file descriptor management, error logging, and redirection mechanisms in Linux systems.